Skill - 22

```
himasree@LAPTOP-41COT5AA:~$ cd ~/OSLab
himasree@LAPTOP-41COT5AA:~/OSLab$ nano redirection.c
himasree@LAPTOP-41COT5AA:~/OSLab$ echo "Hello from the input file!" > input.txt
echo "This is line 2." >> input.txt
echo "This is line 3." >> input.txt
himasree@LAPTOP-41COT5AA:~/OSLab$ cat input.txt
Hello from the input file!
This is line 2.
This is line 3.
himasree@LAPTOP-41COT5AA:~/OSLab$
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc redirection.c -o redirection
himasree@LAPTOP-41COT5AA:~/OSLab$ ./redirection
========================================
       INPUT / OUTPUT REDIRECTION
========================================

Enter input file name: input.txt
Enter output file name: output.txt

--- Input Redirection: input.txt ---
Hello from the input file!
This is line 2.
This is line 3.

--- End of Input ---

========================================
 Redirection demonstration completed.
========================================
himasree@LAPTOP-41COT5AA:~/OSLab$
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat output.txt
This text was redirected to the output file.
Output redirection using dup2() was successful.
himasree@LAPTOP-41COT5AA:~/OSLab$
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat redirection.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define BUFFER_SIZE 1024

void input_redirection(const char *filename)
{
    int fd = open(filename, O_RDONLY);

    if (fd < 0)
    {
        perror("Error opening input file");
        return;
    }

    if (dup2(fd, STDIN_FILENO) < 0)
    {
        perror("dup2 failed");
        close(fd);
        return;
    }

    close(fd);
```

```
    printf("\n--- Input Redirection: %s ---\n", filename);

    char buffer[BUFFER_SIZE];
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buffer, BUFFER_SIZE - 1)) > 0)
    {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    printf("\n--- End of Input ---\n");
}

void output_redirection(const char *filename)
{
    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
    {
        perror("Error creating output file");
        return;
    }

    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 failed");
        close(fd);
        return;
    }
```

```
    printf("This text was redirected to the output file.\n");
    printf("Output redirection using dup2() was successful.\n");
}

int main()
{
    char input_file[100];
    char output_file[100];

    printf("=========================================\n");
    printf("       INPUT / OUTPUT REDIRECTION\n");
    printf("=========================================\n");

    printf("\nEnter input file name: ");
    scanf("%99s", input_file);

    printf("Enter output file name: ");
    scanf("%99s", output_file);

    /*
     * Demonstrate input redirection in a child process.
     * This prevents changing the parent's stdin permanently.
     */
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
```

```
    }

    wait(NULL);

    /*
     * Demonstrate output redirection in another child process.
     */
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        output_redirection(output_file);
        exit(0);
    }

    wait(NULL);

    printf("\n=========================================\n");
    printf(" Redirection demonstration completed.\n");
    printf("=========================================\n");

    return 0;
}
himasree@LAPTOP-41COT5AA:~/OSLab$
```